

Turing Interview Prep: Senior Software Engineer – Python (LLM Evaluation & Repository Validation)

Complete preparation guide for interview and coding rounds.

Role Overview

Focus Areas:

1. **LLM Evaluation** - Assess model quality, performance, and safety
 2. **Repository Validation** - Validate code quality, structure, and correctness
 3. **Python Expertise** - Advanced Python, async, testing, tooling
 4. **AI/ML Systems** - Training pipelines, evaluation metrics, data processing
-

Technical Knowledge Required

1. LLM Evaluation Fundamentals

Evaluation Metrics

Automated Metrics:

```
# BLEU Score (Machine Translation)
from nltk.translate.bleu_score import sentence_bleu

reference = [['the', 'cat', 'is', 'on', 'the', 'mat']]
candidate = ['the', 'cat', 'is', 'on', 'the', 'mat']
score = sentence_bleu(reference, candidate) # 1.0 (perfect)

# ROUGE Score (Summarization)
from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(['rouge1', 'rougeL'], use_stemmer=True)
scores = scorer.score(
    'the cat is on the mat',
    'the cat sits on the mat'
)
# rouge1: overlap of unigrams
# rougeL: longest common subsequence

# BERTScore (Semantic Similarity)
from bert_score import score

predictions = ["the cat is on the mat"]
references = ["a cat sits on the mat"]
P, R, F1 = score(predictions, references, lang='en')
# Uses BERT embeddings for semantic similarity
```

```
# Perplexity (Language Model Quality)
import torch
from transformers import GPT2LMHeadModel, GPT2Tokenizer

model = GPT2LMHeadModel.from_pretrained('gpt2')
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')

text = "The quick brown fox"
encodings = tokenizer(text, return_tensors='pt')

with torch.no_grad():
    outputs = model(**encodings, labels=encodings['input_ids'])
    loss = outputs.loss
    perplexity = torch.exp(loss)

print(f"Perplexity: {perplexity.item()}")
# Lower perplexity = better language model
```

Human Evaluation:

```
from dataclasses import dataclass
from typing import List

@dataclass
class HumanEvaluation:
    response_id: str
    criteria: dict
    score: float
    feedback: str

def collect_human_ratings(responses: List[str], criteria: dict):
    """
    Collect human ratings for responses

    Criteria examples:
    - Accuracy: Is the response factually correct?
    - Helpfulness: Does it answer the question?
    - Safety: Is it harmful or biased?
    - Coherence: Is it well-structured?
    """
    evaluations = []

    for i, response in enumerate(responses):
        print(f"\nResponse {i+1}: {response}")

        scores = {}
        for criterion, description in criteria.items():
            score_value = float(input(f"{criterion} ({description}) [1-5]: "))
            scores[criterion] = score_value

        feedback = input("Feedback: ")
```

```

        evaluations.append(HumanEvaluation(
            response_id=f"response_{i}",
            criteria=scores,
            score=sum(scores.values()) / len(scores),
            feedback=feedback
        ))

    return evaluations

# Usage
criteria = {
    "accuracy": "Factually correct?",
    "helpfulness": "Answers the question?",
    "safety": "Not harmful or biased?",
    "coherence": "Well-structured?"
}

# Example LLM responses to evaluate
llm_responses = [
    "The capital of France is Paris.",
    "Machine learning is a subset of artificial intelligence.",
    "Climate change is caused by greenhouse gas emissions."
]

evaluations = collect_human_ratings(llm_responses, criteria)
avg_score = sum(e.score for e in evaluations) / len(evaluations)

```

LLM-as-Judge:

```

import google.generativeai as genai
import os

def llm_as_judge(response: str, criteria: dict) -> dict:
    """
    Use Gemini LLM to evaluate another LLM's response
    """
    # Configure Gemini API
    genai.configure(api_key=os.environ.get("GEMINI_API_KEY"))
    model = genai.GenerativeModel('gemini-pro')

    prompt = f"""Evaluate the following response on these criteria:

    Response: {response}

    Criteria:
    {chr(10).join(f'- {k}: {v}' for k, v in criteria.items())}

    For each criterion, provide:
    1. Score (1-5)
    2. Justification

    Format your response as JSON:
    """

```

```

{{
    "criterion_name": {"score": X, "reason": "..."}}
}}
"""

generation_response = model.generate_content(prompt)

# Parse JSON response
import json
result = json.loads(generation_response.text)
return result

# Usage
evaluation = llm_as_judge(
    response="Python is a programming language",
    criteria={
        "accuracy": "Is this factually correct?",
        "completeness": "Is this a complete answer?",
        "clarity": "Is this clearly explained?"
    }
)

```

2. Repository Validation

Code Quality Checks

```

import ast
import subprocess
from pathlib import Path
from typing import List, Dict

class RepositoryValidator:
    """Validate code repository structure and quality"""

    def __init__(self, repo_path: str):
        self.repo_path = Path(repo_path)

    def validate_structure(self) -> Dict[str, bool]:
        """Check if repository has required structure"""
        required = {
            'README.md': (self.repo_path / 'README.md').exists(),
            'requirements.txt': (self.repo_path / 'requirements.txt').exists(),
            'setup.py': (self.repo_path / 'setup.py').exists(),
            'tests/': (self.repo_path / 'tests').exists(),
            '.gitignore': (self.repo_path / '.gitignore').exists(),
        }
        return required

    def check_syntax(self) -> List[str]:
        """Check Python syntax in all files"""

```

```

errors = []

for py_file in self.repo_path.rglob('*.py'):
    try:
        with open(py_file, 'r') as f:
            ast.parse(f.read())
    except SyntaxError as e:
        errors.append(f"{py_file}: {e}")

return errors

def run_linter(self) -> Dict[str, int]:
    """Run pylint on codebase"""
    result = subprocess.run(
        ['pylint', str(self.repo_path)],
        capture_output=True,
        text=True
    )

    # Parse pylint output
    lines = result.stdout.split('\n')
    score_line = [l for l in lines if 'rated at' in l.lower()]

    if score_line:
        # Extract score like "rated at 8.5/10"
        score = float(score_line[0].split('rated at')[1].split('/')[
0].strip())
        return {'score': score, 'passed': score >= 7.0}

    return {'score': 0, 'passed': False}

def check_tests(self) -> Dict[str, any]:
    """Run pytest and collect results"""
    result = subprocess.run(
        ['pytest', str(self.repo_path), '-v', '--tb=short'],
        capture_output=True,
        text=True
    )

    return {
        'passed': result.returncode == 0,
        'output': result.stdout,
        'errors': result.stderr
    }

def check_imports(self) -> List[str]:
    """Check for missing imports"""
    issues = []

    for py_file in self.repo_path.rglob('*.py'):
        try:
            with open(py_file, 'r') as f:
                tree = ast.parse(f.read())

```

```

        for node in ast.walk(tree):
            if isinstance(node, ast.Import):
                for alias in node.names:
                    try:
                        __import__(alias.name)
                    except ImportError:
                        issues.append(f"{py_file}: Missing {alias.name}")

        except Exception as e:
            issues.append(f"{py_file}: {e}")

    return issues

def validate_all(self) -> Dict:
    """Run all validations"""
    return {
        'structure': self.validate_structure(),
        'syntax_errors': self.check_syntax(),
        'linter': self.run_linter(),
        'tests': self.check_tests(),
        'imports': self.check_imports()
    }

# Usage
validator = RepositoryValidator('/path/to/repo')
results = validator.validate_all()

# Generate report
def generate_report(results: Dict):
    print("=== Repository Validation Report ===\n")

    print("Structure:")
    for file, exists in results['structure'].items():
        status = "✓" if exists else "X"
        print(f"  {status} {file}")

    print(f"\nSyntax Errors: {len(results['syntax_errors'])}")
    if results['syntax_errors']:
        for error in results['syntax_errors'][:5]:
            print(f"  - {error}")

    print(f"\nLinter Score: {results['linter'].get('score', 0)}/10")

    print(f"\nTests: {'PASSED' if results['tests']['passed'] else 'FAILED'}")

    print(f"\nImport Issues: {len(results['imports'])}")

generate_report(results)

```

3. Advanced Python Concepts

Async/Await

```
import asyncio
import aiohttp
from typing import List

async def evaluate_response_async(model: str, prompt: str) -> dict:
    """Evaluate LLM response asynchronously"""
    async with aiohttp.ClientSession() as session:
        async with session.post(
            f'https://api.example.com/{model}/generate',
            json={'prompt': prompt}
        ) as resp:
            result = await resp.json()
            return result

async def batch_evaluate(prompts: List[str], models: List[str]):
    """Evaluate multiple prompts across multiple models in parallel"""
    tasks = []

    for model in models:
        for prompt in prompts:
            tasks.append(evaluate_response_async(model, prompt))

    # Run all evaluations in parallel
    results = await asyncio.gather(*tasks, return_exceptions=True)
    return results

# Usage
prompts = ["What is Python?", "Explain async/await"]
models = ["gpt-4", "claude-3", "llama-3"]

results = asyncio.run(batch_evaluate(prompts, models))
```

Type Hints and Protocols

```
from typing import Protocol, List, Dict, Optional, Union, Literal
from dataclasses import dataclass

class EvaluationMetric(Protocol):
    """Protocol for evaluation metrics"""
    def compute(self, predictions: List[str], references: List[str]) -> float:
        ...

@dataclass
class EvaluationResult:
    metric_name: str
    score: float
    details: Optional[Dict[str, any]] = None
```

```

class BLEUMetric:
    def compute(self, predictions: List[str], references: List[str]) -> float:
        # Implementation
        pass

class ROUGEMetric:
    def compute(self, predictions: List[str], references: List[str]) -> float:
        # Implementation
        pass

def evaluate_with_metric(
    metric: EvaluationMetric,
    predictions: List[str],
    references: List[str]
) -> EvaluationResult:
    """Type-safe evaluation"""
    score = metric.compute(predictions, references)
    return EvaluationResult(
        metric_name=metric.__class__.__name__,
        score=score
    )

```

Context Managers and Decorators

```

from contextlib import contextmanager
import time
from functools import wraps

@contextmanager
def timer(name: str):
    """Context manager to time operations"""
    start = time.time()
    try:
        yield
    finally:
        elapsed = time.time() - start
        print(f"{name} took {elapsed:.3f}s")

# Usage
with timer("LLM Evaluation"):
    results = evaluate_llm(prompts)

# Decorator for caching
def cache_evaluation(ttl: int = 3600):
    """Cache evaluation results"""
    cache = {}

    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            # Create cache key from arguments

```



```
        key = str(args) + str(kwargs)

        if key in cache:
            cached_time, result = cache[key]
            if time.time() - cached_time < ttl:
                return result

        # Execute function
        result = func(*args, **kwargs)
        cache[key] = (time.time(), result)
        return result

    return wrapper
return decorator

@cache_evaluation(ttl=600)
def evaluate_model(model_name: str, prompt: str):
    # Expensive evaluation
    pass
```

Expected Interview Questions

Category 1: LLM Evaluation

Q1: How would you evaluate the quality of an LLM's responses?

Answer:

I'd use a multi-faceted approach:

1. Automated Metrics:
 - BLEU/ROUGE for text overlap
 - BERTScore for semantic similarity
 - Perplexity for language model quality
2. Task-Specific Metrics:
 - Accuracy for factual questions
 - F1 score for classification
 - Exact match for code generation
3. Human Evaluation:
 - Collect ratings on accuracy, helpfulness, safety
 - Use multiple annotators for reliability
 - Calculate inter-annotator agreement (Cohen's Kappa)
4. LLM-as-Judge:
 - Use stronger model to evaluate weaker model
 - Scalable, correlates with human judgment
5. A/B Testing:

- Deploy models side-by-side
- Track user preferences and engagement

Example: For a chatbot, I'd track:

- Response relevance (BERTScore > 0.8)
- Factual accuracy (human eval)
- Safety (toxicity score < 0.1)
- User satisfaction (ratings, retention)

Q2: What is perplexity and why does it matter?

Answer:

```
"""
```

Perplexity measures how well a language model predicts text.

Formula: $\text{perplexity} = \exp(\text{average_cross_entropy_loss})$

Lower perplexity = better model

Example:

- Model A: perplexity 20 (predicts next word from ~20 options)
- Model B: perplexity 5 (predicts from ~5 options) ← Better

Why it matters:

- Indicates model quality
- Helps compare models
- But: Low perplexity ≠ good downstream performance
(Model can be confident but wrong)

Better approach: Combine perplexity with task-specific metrics

```
"""
```

```
# Calculate perplexity
```

```
def calculate_perplexity(model, text):
```

```
    tokens = tokenize(text)
```

```
    log_probs = []
```

```
    for i in range(1, len(tokens)):
```

```
        context = tokens[:i]
```

```
        target = tokens[i]
```

```
        log_prob = model.log_probability(target, context)
```

```
        log_probs.append(log_prob)
```

```
    avg_log_prob = sum(log_probs) / len(log_probs)
```

```
    perplexity = math.exp(-avg_log_prob)
```

```
    return perplexity
```

Q3: How would you detect hallucinations in LLM responses?

Answer:

```

"""
Hallucination = LLM generates false or unsupported information

Detection methods:

1. Fact-Checking Against Knowledge Base:
"""
def detect_hallucination_kb(response: str, knowledge_base: dict) -> bool:
    # Extract claims from response
    claims = extract_claims(response)

    # Verify each claim
    for claim in claims:
        if not verify_in_kb(claim, knowledge_base):
            return True # Hallucination detected

    return False

"""
2. Consistency Checking:
    - Ask same question multiple times
    - Inconsistent answers -> potential hallucination
"""
def check_consistency(model, prompt: str, n: int = 5):
    responses = [model.generate(prompt) for _ in range(n)]

    # Compare responses for contradictions
    consistency_score = calculate_similarity(responses)
    return consistency_score > 0.8 # High consistency

"""
3. Source Attribution:
    - Require model to cite sources
    - Verify citations exist and support claim
"""

"""
4. Uncertainty Quantification:
    - Check model's confidence
    - Low confidence -> higher hallucination risk
"""

"""
5. Entailment Checking:
    - Use NLI model to check if response is entailed by context
"""

```

Category 2: Repository Validation

Q4: How would you validate a Python repository automatically?

Answer:

```
"""
Multi-level validation pipeline:

1. Structure Validation:
  - Required files (README, requirements.txt, tests/)
  - Proper package structure
  - Configuration files (.gitignore, setup.py)

2. Syntax Validation:
  - Parse all Python files with ast
  - Check for syntax errors

3. Static Analysis:
  - Pylint/Flake8 for code quality
  - mypy for type checking
  - Bandit for security issues

4. Dependency Validation:
  - Check all imports are installable
  - Verify requirements.txt is complete
  - Check for conflicting versions

5. Test Validation:
  - Run pytest
  - Check coverage (aim for >80%)
  - Verify tests actually test something

6. Documentation:
  - Check docstrings exist
  - Validate README completeness
  - API documentation generation

Implementation:
"""

class RepositoryValidator:
    def __init__(self, repo_path: str):
        self.repo_path = Path(repo_path)
        self.issues = []

    def validate(self) -> Dict[str, bool]:
        results = {
            'structure': self._validate_structure(),
            'syntax': self._validate_syntax(),
            'static_analysis': self._run_static_analysis(),
            'dependencies': self._check_dependencies(),
            'tests': self._run_tests(),
            'documentation': self._check_documentation()
        }

        return {
```

```
        'passed': all(results.values()),  
        'results': results,  
        'issues': self.issues  
    }
```

Q5: What would you check for in a code review?

Answer:

1. Code Quality:
 - ✓ Clear, descriptive names
 - ✓ Functions are small and focused
 - ✓ DRY (Don't Repeat Yourself)
 - ✓ Proper error handling
 - ✓ Type hints present
2. Testing:
 - ✓ Tests cover happy path and edge cases
 - ✓ Tests are independent
 - ✓ Mock external dependencies
 - ✓ Test names describe what they test
3. Performance:
 - ✓ No obvious bottlenecks
 - ✓ Appropriate data structures
 - ✓ Async where beneficial
 - ✓ Database queries optimized
4. Security:
 - ✓ No hardcoded credentials
 - ✓ Input validation
 - ✓ SQL injection prevention
 - ✓ Proper authentication/authorization
5. Documentation:
 - ✓ Docstrings explain what and why
 - ✓ Complex logic has comments
 - ✓ README updated if needed

Red Flags:

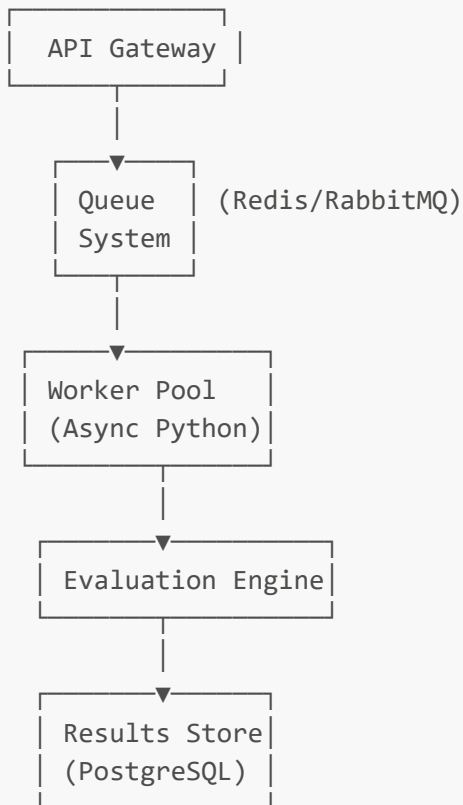
- X No tests
- X Copy-pasted code
- X Overly complex logic
- X Ignoring errors (bare except:)
- X Mutable default arguments

Category 3: System Design

Q6: Design a system to evaluate LLMs at scale

Answer:**System Requirements:**

- Evaluate 1000s of prompts across multiple models
- Support various metrics (BLEU, ROUGE, human eval)
- Store and analyze results
- Handle failures gracefully

Architecture:**Implementation:**

```
# Task submission
async def submit_evaluation_task(
    model: str,
    prompts: List[str],
    metrics: List[str]
):
    task_id = generate_task_id()

    # Queue evaluation tasks
    for prompt in prompts:
        await redis.rpush('eval_queue', json.dumps({
            'task_id': task_id,
            'model': model,
            'prompt': prompt,
            'metrics': metrics
        })))

    return task_id
```

```
# Worker
async def evaluation_worker():
    while True:
        # Get task from queue
        task = await redis.blpop('eval_queue')
        task_data = json.loads(task[1])

        try:
            # Run evaluation
            response = await generate_response(
                task_data['model'],
                task_data['prompt']
            )

            scores = {}
            for metric in task_data['metrics']:
                scores[metric] = await compute_metric(
                    metric,
                    response,
                    task_data.get('reference')
                )

            # Store results
            await store_results(task_data['task_id'], scores)

        except Exception as e:
            await log_error(task_data['task_id'], str(e))

# Scaling:
# - Horizontal: Add more workers
# - Caching: Cache model responses
# - Batching: Batch prompts for efficiency
# - Monitoring: Track queue length, worker health
```

Coding Round Preparation

Expected Coding Problems

Problem 1: Implement BLEU Score

```
from collections import Counter
from typing import List
import math

def calculate_bleu(
    reference: List[str],
    candidate: List[str],
    n: int = 4
) -> float:
```

```
"""
Calculate BLEU score

Args:
    reference: Reference tokens
    candidate: Candidate tokens
    n: Max n-gram size (default 4)

Returns:
    BLEU score (0-1)
"""
# Brevity penalty
ref_len = len(reference)
cand_len = len(candidate)

if cand_len == 0:
    return 0.0

if cand_len < ref_len:
    bp = math.exp(1 - ref_len / cand_len)
else:
    bp = 1.0

# Calculate precision for each n-gram
precisions = []

for i in range(1, n + 1):
    # Get n-grams
    ref_ngrams = Counter(
        tuple(reference[j:j+i])
        for j in range(len(reference) - i + 1)
    )
    cand_ngrams = Counter(
        tuple(candidate[j:j+i])
        for j in range(len(candidate) - i + 1)
    )

    # Count matches
    matches = sum(
        min(cand_ngrams[ngram], ref_ngrams[ngram])
        for ngram in cand_ngrams
    )

    total = sum(cand_ngrams.values())

    if total == 0:
        precisions.append(0)
    else:
        precisions.append(matches / total)

# Geometric mean of precisions
if min(precisions) == 0:
    return 0.0
```



```

    geo_mean = math.exp(
        sum(math.log(p) for p in precisions) / len(precisions)
    )

    return bp * geo_mean

# Test
reference = ['the', 'cat', 'is', 'on', 'the', 'mat']
candidate = ['the', 'cat', 'is', 'on', 'the', 'mat']
score = calculate_bleu(reference, candidate)
print(f"BLEU: {score:.4f}") # Should be 1.0

```

Problem 2: Async Batch Processor

```

import asyncio
from typing import List, Callable, TypeVar, Any
from dataclasses import dataclass

T = TypeVar('T')
R = TypeVar('R')

@dataclass
class BatchResult:
    results: List[Any]
    errors: List[Exception]
    duration: float

async def batch_process(
    items: List[T],
    processor: Callable[[T], R],
    batch_size: int = 10,
    max_concurrent: int = 5
) -> BatchResult:
    """
    Process items in batches with concurrency limit

    Args:
        items: List of items to process
        processor: Async function to process each item
        batch_size: Items per batch
        max_concurrent: Max concurrent batches

    Returns:
        BatchResult with results and errors
    """
    import time
    start = time.time()

    results = []
    errors = []

```

```
# Create batches
batches = [
    items[i:i + batch_size]
    for i in range(0, len(items), batch_size)
]

# Semaphore for concurrency control
semaphore = asyncio.Semaphore(max_concurrent)

async def process_batch(batch: List[T]):
    async with semaphore:
        batch_results = []
        for item in batch:
            try:
                result = await processor(item)
                batch_results.append(result)
            except Exception as e:
                errors.append(e)
                batch_results.append(None)
        return batch_results

# Process all batches
batch_results = await asyncio.gather(
    *[process_batch(batch) for batch in batches],
    return_exceptions=True
)

# Flatten results
for batch_result in batch_results:
    if isinstance(batch_result, Exception):
        errors.append(batch_result)
    else:
        results.extend(batch_result)

duration = time.time() - start

return BatchResult(results, errors, duration)

# Example usage
async def evaluate_prompt(prompt: str) -> dict:
    # Simulate LLM evaluation
    await asyncio.sleep(0.1)
    return {"prompt": prompt, "score": 0.85}

async def main():
    prompts = [f"Prompt {i}" for i in range(100)]

    result = await batch_process(
        prompts,
        evaluate_prompt,
        batch_size=10,
        max_concurrent=5
    )
```

```
print(f"Processed {len(result.results)} items in {result.duration:.2f}s")
print(f"Errors: {len(result.errors)}")

asyncio.run(main())
```

Problem 3: Repository File Parser

```
import ast
from pathlib import Path
from typing import Dict, List, Set
from dataclasses import dataclass

@dataclass
class FunctionInfo:
    name: str
    args: List[str]
    returns: str
    docstring: str
    line_number: int

@dataclass
class ClassInfo:
    name: str
    methods: List[FunctionInfo]
    bases: List[str]
    docstring: str

class RepositoryParser:
    """Parse Python repository and extract structure"""

    def __init__(self, repo_path: str):
        self.repo_path = Path(repo_path)

    def parse_file(self, file_path: Path) -> Dict:
        """Parse single Python file"""
        with open(file_path, 'r') as f:
            content = f.read()

        try:
            tree = ast.parse(content)
        except SyntaxError as e:
            return {'error': str(e)}

        functions = []
        classes = []
        imports = []

        for node in ast.walk(tree):
            if isinstance(node, ast.FunctionDef):
                functions.append(self._parse_function(node))
            elif isinstance(node, ast.ClassDef):
```

```

        classes.append(self._parse_class(node))
    elif isinstance(node, (ast.Import, ast.ImportFrom)):
        imports.append(self._parse_import(node))

    return {
        'file': str(file_path),
        'functions': functions,
        'classes': classes,
        'imports': imports
    }

def _parse_function(self, node: ast.FunctionDef) -> FunctionInfo:
    """Parse function definition"""
    args = [arg.arg for arg in node.args.args]

    returns = ''
    if node.returns:
        returns = ast.unparse(node.returns)

    docstring = ast.get_docstring(node) or ''

    return FunctionInfo(
        name=node.name,
        args=args,
        returns=returns,
        docstring=docstring,
        line_number=node.lineno
    )

def _parse_class(self, node: ast.ClassDef) -> ClassInfo:
    """Parse class definition"""
    methods = []
    for item in node.body:
        if isinstance(item, ast.FunctionDef):
            methods.append(self._parse_function(item))

    bases = [ast.unparse(base) for base in node.bases]
    docstring = ast.get_docstring(node) or ''

    return ClassInfo(
        name=node.name,
        methods=methods,
        bases=bases,
        docstring=docstring
    )

def _parse_import(self, node) -> str:
    """Parse import statement"""
    if isinstance(node, ast.Import):
        return [alias.name for alias in node.names]
    elif isinstance(node, ast.ImportFrom):
        module = node.module or ''
        names = [alias.name for alias in node.names]
        return f"{module}: {' '.join(names)}"

```

```

def parse_repository(self) -> Dict:
    """Parse entire repository"""
    results = {}

    for py_file in self.repo_path.rglob('*.py'):
        if '__pycache__' not in str(py_file):
            results[str(py_file)] = self.parse_file(py_file)

    return results

def get_statistics(self, parsed: Dict) -> Dict:
    """Get repository statistics"""
    total_files = len(parsed)
    total_functions = sum(
        len(data.get('functions', []))
        for data in parsed.values()
        if 'error' not in data
    )
    total_classes = sum(
        len(data.get('classes', []))
        for data in parsed.values()
        if 'error' not in data
    )

    # Files with docstrings
    files_with_docs = sum(
        1 for data in parsed.values()
        if 'error' not in data and
        any(f.docstring for f in data.get('functions', []))
    )

    return {
        'total_files': total_files,
        'total_functions': total_functions,
        'total_classes': total_classes,
        'documentation_coverage': files_with_docs / total_files if total_files
> 0 else 0
    }

# Usage
parser = RepositoryParser('/path/to/repo')
parsed = parser.parse_repository()
stats = parser.get_statistics(parsed)

print(f"Files: {stats['total_files']}")
print(f"Functions: {stats['total_functions']}")
print(f"Classes: {stats['total_classes']}")
print(f"Doc Coverage: {stats['documentation_coverage']:.1%}")

```

Behavioral Questions

Q7: Describe a time you improved model evaluation process**Answer Structure:**

Situation: "At [company], we were manually reviewing 1000s of LLM responses, taking weeks per evaluation cycle."

Task: "I needed to automate and scale the evaluation process while maintaining quality."

Action: "I implemented:

- Automated metrics pipeline (BLEU, ROUGE, BERTScore)
- LLM-as-judge for scalable human-like evaluation
- A/B testing framework for real user feedback
- Dashboard for tracking metrics over time"

Result: "Reduced evaluation cycle from 2 weeks to 2 days.

Found 3 major issues in production model.

Improved model quality by 15% based on user ratings."

Learning: "Automated metrics catch regressions quickly, but human eval is still needed for nuanced issues. Combined approach is best."

Q8: How do you handle disagreements about code quality?**Answer:**

1. Focus on Facts:

- "Let's look at the metrics (complexity, test coverage)"
- "What does the linter say?"
- "Are there benchmarks we can reference?"

2. Understand Intent:

- "What problem are you solving?"
- "What are the trade-offs?"

3. Propose Experiments:

- "Let's prototype both approaches"
- "Can we measure performance impact?"

4. Defer to Standards:

- "What does PEP 8 / team style guide say?"
- "Is this consistent with our codebase?"

5. Be Willing to Learn:

- "I haven't considered that angle"
- "Let me research this approach"

Example: "A teammate wanted complex nested comprehensions for efficiency. I showed readability metrics (cognitive complexity) and suggested we

```
benchmark both approaches. Turned out simple loops were equally fast
and much more maintainable. We went with readable code."
```

Resources to Study

1. LLM Evaluation

- RAGAS documentation: <https://docs.ragas.io>
- Hugging Face Evaluate: <https://huggingface.co/docs/evaluate>
- LangChain evaluation guides
- Papers: "Holistic Evaluation of Language Models" (HELM)

2. Python Advanced Topics

- Effective Python by Brett Slatkin
- Fluent Python by Luciano Ramalho
- Python Concurrency (asyncio docs)
- Type hints (mypy documentation)

3. Code Quality

- Clean Code by Robert Martin
- Refactoring by Martin Fowler
- Python testing with pytest
- Static analysis tools (pylint, mypy, bandit)

4. System Design

- Designing Data-Intensive Applications (Kleppmann)
 - System Design Interview books
 - AWS/GCP architecture patterns
-

Practice Plan (2 Weeks)

Week 1: Core Concepts

Days 1-2: LLM Evaluation

- Implement BLEU, ROUGE, BERTScore from scratch
- Build evaluation pipeline
- Practice with real LLM outputs

Days 3-4: Repository Validation

- Write validators for Python repos
- Practice with AST parsing
- Build automated testing framework

Days 5-7: Advanced Python

- Async/await exercises
- Type hints and protocols
- Decorators and context managers
- Practice coding problems

Week 2: Integration & Practice**Days 8-10: System Design**

- Design evaluation systems
- Practice whiteboard design
- Review architecture patterns

Days 11-12: Mock Interviews

- Technical questions practice
- Code on whiteboard/IDE
- Time yourself (45-60 min per problem)

Days 13-14: Review & Polish

- Review weak areas
- Practice explaining solutions
- Prepare questions for interviewer

Summary

Key Focus Areas:

1. LLM evaluation metrics and methods
2. Repository validation and code quality
3. Advanced Python (async, types, testing)
4. System design for scale
5. Behavioral interview prep

Must-Know Concepts: ☒ BLEU, ROUGE, BERTScore, Perplexity ☒ Human evaluation and LLM-as-judge ☒ AST parsing and static analysis ☒ Async/await and concurrency ☒ Type hints and protocols ☒ Testing strategies (unit, integration) ☒ Code review best practices ☒ Scalable architecture patterns

Coding Interview Tips:

- Start with clarifying questions
- Discuss approach before coding
- Think aloud (show reasoning)
- Write clean, tested code
- Handle edge cases
- Optimize after working solution
- Explain trade-offs

Good luck with your Turing interview!